

Audio Corpus Feature Preprocessor

User Manual

A Max tool for preparing audio descriptor corpora for distance-based navigation, clustering, and nearest-neighbor queries. Designed to work with the FluCoMa toolkit.

Components: preprocess.js (computation) · diagnostics_ui.js (visual panel) · HTML report (jweb)

1. Overview

The Audio Corpus Feature Preprocessor takes raw audio descriptor data and transforms it so that features become comparable, well-behaved, and useful for distance-based operations. Raw audio features have mismatched scales, skewed distributions, outliers, and redundancies that corrupt distance calculations. This tool fixes these issues in a controlled pipeline.

Architecture

The pipeline runs four steps in preprocess.js, then hands off to FluCoMa for normalization and spatial operations:

```
Raw corpus → 1. Impute → 2. Correlate → 3. Log → 4. Winsorize → [output JSON]
↓
fluid.standardize~ → fluid.umap~ → fluid.kdtree~
```

File Structure

```
your-project/
javascript/
preprocess.js (main script – runs in [js])
diagnostics_ui.js (visual panel – runs in [jsui])
data/
corpus_raw.json (input)
corpus_processed.json (output – load into fluid.dataset~)
transform_params.json (output – for runtime normalization)
diagnostics.html (output – visual report for [jweb])
```

2. Installation

1. Copy preprocess.js and diagnostics_ui.js into a folder visible to Max (same folder as your patch, or listed in Max's File Preferences search path).

2. Create the objects in your Max patch:

```
[js preprocess.js]
[jsui @filename diagnostics_ui.js @size 620 520]
```

3. Wire the outlets:

Outlet	Content	Connect to
0	Status messages	[route loaded mode committed error preview_done report_written]
1	Diagnostics dict name	[jsui] inlet (triggers panel redraw)
2	Bang (after commit)	[fluid.dataset~] load message

3. Pipeline Steps

Step 1: Missing Value Imputation

Purpose: Fill gaps in your feature data. Some audio analyzers produce NaN when they cannot extract a meaningful value — for example, a pitch tracker applied to noise.

Method: Scans every feature column for NaN, null, or a user-defined sentinel value. Replaces each missing value with the median of that feature across all valid samples.

When to use: Always enable as a first pass. Even if you believe your data is clean, detection costs nothing and catches hidden problems.

Parameter	Type	Default	Range	Description
nan_sentinel	float	none	any number	Treat this value as missing (e.g. 0, -1)
fill_values	float array	median	any numbers	Manual fill values, one per feature

```
set_mode impute auto
set_param impute nan_sentinel 0
```

Step 2: Correlation-Based Feature Selection

Purpose: Remove redundant features. If Brightness and Spectral Centroid are correlated at $r = 0.91$, keeping both double-counts that timbral dimension in distance calculations.

Method: Computes the Pearson correlation matrix. For each pair above the threshold, drops the feature with higher average correlation to all other features (the more redundant one). Uses a greedy iterative algorithm.

Parameter	Type	Default	Range	Description
threshold	float	0.85	0.0 – 1.0	Correlation above which a pair is flagged
keep_indices	int array	all	feature indices	Manual: which original indices to keep
prefer_indices	int array	none	feature indices	Auto: prefer keeping these features

```
set_mode correlate auto
set_param correlate threshold 0.9
set_param correlate prefer_indices 0 3
```

Step 3: Log Transform

Purpose: Compress features with long-tailed distributions. Audio features like Spectral Flux often have most values near zero with a few extreme values stretching the range by orders of magnitude.

Method: Applies the signed log1p transform: $y = \text{sign}(x) \times \log(1 + |x|)$. This compresses large values exponentially while barely affecting small values. The transform is monotonic (preserves rank order) and invertible.

Note: The log transform distorts probability densities via the Jacobian factor. If your system uses density-based similarity, be aware that rare extreme values will appear artificially dense in the transformed space.

Parameter	Type	Default	Range	Description
skew_threshold	float	2.0	≥ 0	Auto: apply log when skewness exceeds this
indices	int array	auto	feature indices	Manual: which original indices to log-transform

```
set_mode log manual
preview ;; see suggestions
set_param log indices 1 4 ;; apply to features 1 and 4
```

Step 4: Winsorization

Purpose: Clip remaining outliers to boundary percentiles. Even after log-transform, some extreme values may persist. Winsorization caps these extremes without removing samples.

Method: For each feature, computes the value at the lower and upper percentile boundaries. Clips all values outside these boundaries to the boundary value. Named after Charles P. Winsor (1940s).

Parameter	Type	Default	Range	Description
lo_pct	float	2	0 – 50	Lower percentile for clipping
hi_pct	float	98	50 – 100	Upper percentile for clipping
boundaries	[lo, hi] array	auto	any numbers	Manual: exact clip boundaries per feature

4. Step Modes

Every step operates in one of three modes:

Mode	Behavior	Use when
off	Step is skipped entirely	You don't need this processing
auto	Runs using internal heuristics, reports what it did	Quick results with sensible defaults
manual	Analyzes and suggests, applies only user-set parameters	Full control over every decision

5. Complete Message Reference

Loading and State

Message	Arguments	Description
load	<filepath>	Load a raw JSON corpus
reset	(none)	Reset all modes and parameters to defaults
get_modes	(none)	Print current modes to Max console
get_params	<step>	Print parameters of a step

Configuration

Message	Arguments	Description
set_mode	<step> <off auto manual>	Set operating mode
set_param	<step> <key> <value...>	Set a parameter (multiple values space-separated)

Execution

Message	Arguments	Description
preview	(none)	Run pipeline, compute diagnostics, update panel. No files written
commit	<corpus_path> [params_path]	Run pipeline and write output files to disk
report	<path.json>	Write diagnostics JSON + HTML report
report_jweb	(none)	Write temp HTML, output URL from outlet 0

Outlet Messages

Outlet	Message	When
0	loaded <n_samples> <n_features>	After load
0	mode <step> <mode>	After set_mode
0	preview_done	After preview
0	committed <path>	After commit
0	report_written <json> <html>	After report
0	report_url <url>	After report_jweb
0	error <message>	On error

Outlet	Message	When
1	diagnostics <dict_name>	After preview (triggers jsui)
2	bang	After commit (triggers fluid.dataset~ load)

6. Data Format

Input and output use the fluid.dataset~ JSON format:

```
{
  "cols": 10,
  "data": {
    "sample_001": [1299.34, 10230.00, 48.50, ...],
    "sample_002": [2041.22, 8754.31, 12.87, ...]
  }
}
```

Optionally include "feature_names": ["Brightness", ...] for labeled output.

The transform parameters file records every transform applied, for normalizing new sounds at runtime.

7. Visualization

jsui Panel (always visible)

Compact dashboard showing mode indicators, per-feature stats with color coding, correlation heatmap, and suggestions. Updates after each preview.

```
[jsui @filename diagnostics_ui.js @size 620 520]
```

Connect outlet 1 of [js preprocess.js] to the jsui inlet.

HTML Report (on demand, in jweb)

Rich interactive report with distribution histograms, box plots, scatter plots for correlated pairs, parallel coordinates, interactive correlation heatmap, and stats table with sparklines. Generated by the report command.

```
preview
report /path/to/diagnostics.json
;; then in jweb:
url file:///path/to/diagnostics.html
```

8. Usage Workflows

Quick Automatic Processing

```
load corpus_raw.json
set_mode impute auto
set_mode correlate auto
set_mode log auto
set_mode winsor auto
preview
commit corpus_processed.json
```

Step-by-Step Evaluation

Enable one step at a time to understand each transform's impact:

```
load corpus_raw.json
preview ;; baseline
set_mode impute auto
preview ;; imputation only
set_mode log auto
preview ;; imputation + log
set_mode winsor auto
preview ;; all three
commit corpus_processed.json
```

Manual Control with System Guidance

```
load corpus_raw.json
set_mode impute auto
set_mode correlate manual
set_mode log manual
set_mode winsor auto
preview ;; see suggestions
set_param correlate keep_indices 0 1 3 4 5 6 7
set_param log indices 1 4
preview ;; verify
commit corpus_processed.json
```

Excluding Metadata Columns

```
load corpus_raw.json
set_mode correlate manual
set_param correlate keep_indices 0 1 2 3 4 5 6 7 8 9
set_mode log auto
set_mode winsor auto
preview
commit corpus_processed.json
```

9. FluCoMa Integration

After committing, load the output into `fluid.dataset~` for normalization and queries:

```
[js preprocess.js] outlet 2 (bang)
→ [message: load corpus_processed.json]
→ [fluid.dataset~ @name preprocessed]
→ [fluid.standardize~ @name scaler]
→ [fluid.dataset~ @name normalized]
→ [fluid.kdtree~ @name tree]
```

For runtime normalization of a new sound, use the stored transform parameters: apply imputation medians, select kept features, apply log1p to logged features, clip to winsor boundaries, then use fluid.standardize~ transform.

10. Troubleshooting

Problem	Solution
Cannot open file	Use filename only (not full path) if file is in patch folder. Check File Preferences.
Run preview first	commit and report require preview to be run first.
No features transformed	In auto mode, lower skew_threshold. In manual mode, set indices.
Too many features dropped	Raise threshold (e.g. 0.95). Use prefer_indices to protect features.
HTML report not generated	Check Max console. Try report_jweb as alternative.
jsui shows no data	Run preview. Check outlet 1 is connected. Send bang to jsui.